

# BabyBlue × ApIOS — Restructure & Implementation Spec

**Status:** Draft v1 · 2026-08-12 · for partner review

**Authors:** Thabo Mollo (+ Claude)

**Scope:** The product repositioning from a supply-side walk-in queue tool into a demand-side healthcare booking marketplace, and the architecture + delivery plan to get there.

## 1. Executive summary

Today the product (ClinicOS) enters from the **supply side**: a clinic buys it, prints a QR poster, and a walk-in patient scans it to see their position at *that one clinic*. The entry point is a physical poster, the scope is a single clinic, and a patient is literally modelled as belonging to one clinic.

We are **inverting the entry point to the demand side**. A patient starts by *searching across all practices* — via Google, an LLM, or by coming to us directly — picks a practitioner, and books. Everything that follows (confirmation, live queue, arrival, consultation) runs on the presence engine we have already built.

This is **not a rewrite**. It is:

**Add a demand-side front door (discovery + booking) and a phone-native runtime (WhatsApp) onto the presence engine we already have — and make one entity, the *Visit*, the spine both halves hang off.**

### Three decisions are locked (2026-08-12):

1. **Hybrid booking.** Practices that configure capacity get real-time slot booking; every other practice falls back to request-to-confirm. This lets us onboard supply immediately without waiting on calendar/PMS integrations, and upgrade each practice individually.
2. **Two front doors, one queue.** The zero-friction QR walk-in path *and* the new online-booking path both create Visits that merge into a single live queue.
3. **Brand & spine split.** **BabyBlue** is the consumer brand (BabyBlue.com — discovery, booking, WhatsApp). **ApIOS** is the renamed platform/spine (formerly ClinicOS) and the practice-facing product. One backend, two branded surfaces.

## 2. Positioning & naming

| Layer                       | Name                             | Audience  | Responsibility                                                  |
|-----------------------------|----------------------------------|-----------|-----------------------------------------------------------------|
| Consumer face               | <b>BabyBlue</b> (BabyBlue.com)   | Patients  | Search, book, WhatsApp queue & presence                         |
| Platform / practice product | <b>ApIOS</b> (was ClinicOS)      | Practices | Practice portal, queue ops, flow analytics; the backend + API   |
| Shared package              | @aplos/core (was @clinicos/core) | Both      | Domain types, Visit state machine, identity, consent, retention |

**Who pays:** Patients never pay to use BabyBlue. Practices pay ApIOS (per-practice subscription). Every feature is scored against: *does it make patients insist their practice uses this, or make the practice's subscription easier to justify?*

## 3. The user journey

The end-to-end patient journey, mapped to the surface that serves each step:

| #  | Step                                                                                                                            | Surface                           |
|----|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| 1  | Patient needs a practitioner/practice                                                                                           | —                                 |
| 2  | Finds BabyBlue via Google, an LLM recommendation, or direct                                                                     | <b>Discovery</b> (SEO / GEO)      |
| 3  | Lands on BabyBlue.com                                                                                                           | <b>Discovery</b>                  |
| 4  | Searches by doctor name, practice type, service, or location                                                                    | <b>Discovery</b> (search)         |
| 5  | Sees results                                                                                                                    | <b>Discovery</b>                  |
| 6  | Starts a booking; enters name, surname, WhatsApp number, reason for visit ( <i>optional immediate payment later</i> )           | <b>Booking</b>                    |
| 7  | Selects date and timeslot                                                                                                       | <b>Booking</b>                    |
| 8  | Reviews details                                                                                                                 | <b>Booking</b>                    |
| 9  | Confirms the booking → <b>a Visit is created</b>                                                                                | <b>Booking</b>                    |
| 10 | Receives WhatsApp confirmation; if within the 2-hour window, gets a live queue read + queue number, and updates as they move up | <b>WhatsApp + Presence</b>        |
| 11 | Arrives and is prompted to confirm arrival                                                                                      | <b>WhatsApp / Presence</b>        |
| 12 | Practice is notified of arrival and confirms it                                                                                 | <b>Practice portal</b>            |
| 13 | Patient goes into the room; consultation proceeds                                                                               | <b>Practice portal</b>            |
| 14 | Consultation concludes                                                                                                          | —                                 |
| 15 | Practice confirms the consultation ended → patient is prompted to confirm                                                       | <b>Practice portal + WhatsApp</b> |

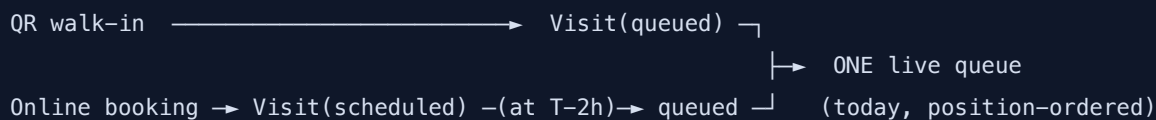
**Key insight:** after booking, **WhatsApp is where the patient lives**. They may never reopen the website. Steps 10–15 are designed *WhatsApp-first*, with the thin web presence page as a fallback reached via magic links.

## 4. Architecture — the Visit spine

### 4.1 One Visit, two front doors, one queue

The unifying primitive is the **Visit** (a generalisation of today's **Appointment**). It has two ways in that converge on one queue:

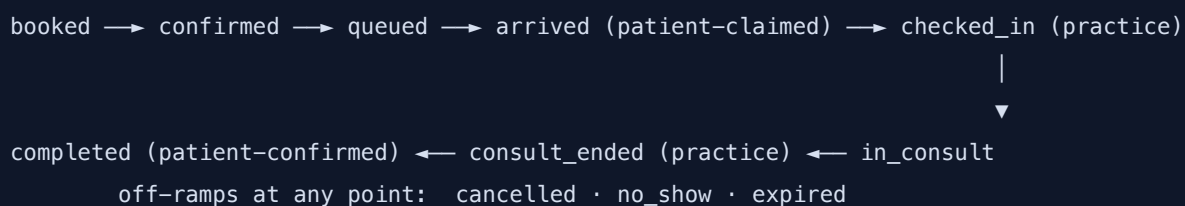
- **Walk-in** (QR scan — today's path) → enters directly as **queued**.
- **Booking** (search → book — the new path) → enters as **scheduled**, and **is promoted into the queue when it crosses the 2-hour window**.



The **2-hour window is the boundary between a calendar-shaped booking** (future, has a slot) **and a position-shaped queue entry** (today, has a number). A promoter job flips a booking to **queued** at window-open; request-mode bookings only promote once **confirmed**. That single rule is the entire "merge booked + walk-in" mechanism.

## 4.2 The Visit state machine

Today's statuses are **scheduled | waiting | in\_consultation | done | cancelled**. The journey requires a richer lifecycle:



**The double-confirmation is deliberate, not redundant.** The patient *claims* arrival and the practice *confirms* it (11→12); the practice *ends* the consult and the patient *confirms* (15). The gap between each paired timestamp is exactly the flow data our tagline promises:

- **True wait** = **queued → checked\_in**
- **True consultation length** = **in\_consult → consult\_ended**

This is "the flow data your practice has never had," produced as a by-product of the confirmation UX.

## 4.3 Hybrid booking — how a practice picks its mode

Each practice carries a **booking\_mode** flag:

- **live** — practice has configured capacity (operating hours + slot length + per-practitioner capacity). Patient sees real open slots → books → instant **confirmed**. Optional calendar/PMS sync can be layered on later.
- **request** (default on onboarding) — no capacity configured. Patient submits a preferred date/time → practice **accepts** or **offers an alternative** → WhatsApp confirms on accept.

The Visit, the funnel UI, and the queue are identical in both modes — only the *confirmation timing* differs. A practice upgrades **request → live** simply by filling in its capacity. This is what lets us onboard supply on day one without blocking on integrations.

# 5. Surfaces & runtimes

The product becomes **three web surfaces, a WhatsApp runtime, and the practice portal** — deliberately separated because they have genuinely different rendering needs.

| Surface         | Journey steps | Rendering need                                         | Notes                                         |
|-----------------|---------------|--------------------------------------------------------|-----------------------------------------------|
| Discovery       | 1–5           | Server-rendered (SSR/SSG), crawlable, machine-readable | Where SEO <b>and</b> LLM-linkability live     |
| Booking         | 6–9           | Interactive, conversion-tuned                          | The funnel; creates the Visit                 |
| Presence        | 10–11         | Real-time, thin                                        | The live queue page, generalised              |
| WhatsApp        | 10, 11, 15    | Event-driven messages + magic links                    | <b>The real patient runtime after booking</b> |
| Practice portal | 12, 13, 15    | Authenticated app                                      | ApLOS admin — arrivals, queue ops, flow data  |

## 6. The four seams that are expensive to reverse

Everything else can be built incrementally. These four are the foundations that are painful to change later, so they must be designed correctly up front.

### Seam 1 — Global patient identity vs. clinic-scoped

Today a `Patient` carries `clinic_id`; a patient *belongs to a clinic*. A marketplace patient is **global**: the WhatsApp number is the durable, cross-practice identity. We must split:

- **Global identity** — `patients` keyed by phone number (the WhatsApp number). One record per human, spanning all practices.
- **Per-practice record** — `practice_patients` holds the clinical file, consent, and POPIA scope for that patient *at that practice*.

**Same phone at Practice A and Practice B must never leak the clinical file across them.** Global identity, per-practice records, consent-scoped access. This is the seam that also unblocks cross-practice search, so it is done first.

### Seam 2 — Visit state machine as pure logic in `@aplos/core`

Transitions and guards live in one place, so all five surfaces agree on what is legal. Types already live in core; the *transitions* join them.

### Seam 3 — Event-driven notification outbox

State transitions emit **events**; a dispatcher maps `event → message → channel`; idempotency is keyed on `(visit_id, event)`. This generalises the existing idempotent notifications table and SMS-provider seam to add the WhatsApp channel and throttled "you've moved up the queue" updates cleanly.

### Seam 4 — Discovery URL structure & structured data

Canonical, hierarchical, indexable URLs (e.g. `/dr/jane-smith/dermatology/johannesburg`) plus schema.org structured data (`Physician`, `MedicalClinic`), a sitemap, and an `llms.txt`. **LLM + SEO discoverability is an architecture requirement, not marketing** — and it must be decided before we have inbound links we cannot afford to break.

---

## 7. Codebase restructure

---

### 7.1 Target monorepo layout ( `clnicos-platform` → `aplos-platform` )

```
aplos-platform/
├─ packages/
│   └─ core/           @aplos/core – types, Visit state machine, identity, consent, retention
├─ apps/
│   └─ api/            Hono service – the single write path; emits events
│       └─ discovery/  BabyBlue SSR/SSG – search + practice/practitioner/service pages
│       └─ patient/    BabyBlue consumer – booking funnel + presence page
│       └─ admin/      AplOS practice portal
│       └─ notifications/ Event-driven dispatcher (or a worker inside api)
```

### 7.2 Rename first

The **ClinicOS** → **AplOS** rename (packages, domains, copy, `@clnicos/core` → `@aplos/core`) is cheap if done *before* the in-flight app-absorb (subtree merge) work, and painful after. Sequence it first.

### 7.3 The API is the single write path

All Visit state transitions go through the Hono API — token-scoped for patients ( `visit_id` + `access_token` ), Bearer-JWT for staff. The API holds the only copy of the service-role key and emits events on every transition. This is already the established direction.

---

## 8. Notifications & events

---

- Every state transition emits a typed event ( `visit.confirmed` , `visit.queued` , `visit.almost_up` , `visit.checked_in` , `visit.consult_ended` , ...).
- A dispatcher maps events to messages and delivers them over a channel behind a provider interface (**SMS today** → **WhatsApp** as the primary SA channel).
- Idempotency is enforced by a send-log keyed on ( `visit_id` , `event` ) — the same message is never sent twice.
- "Updated whenever they move up the queue" is a **throttled** position-change notification (e.g. only on crossing key thresholds: position ≤ 3, position = 1 / "you're next").

WhatsApp specifics: use the WhatsApp Business API with interactive templates (buttons for "Confirm arrival", "I've left the queue") and magic links back to the thin presence page. Approval lead-time for WhatsApp templates should be started early.

---

## 9. Discoverability (SEO + GEO)

---

"A patient asks an LLM and gets a BabyBlue link" is a concrete engineering requirement:

- **Server-rendered** practitioner / practice / service / location pages.
  - **Canonical, hierarchical URLs** per practitioner + service + location.
  - **schema.org structured data** ( `Physician` , `MedicalClinic` , `MedicalSpecialty` ) for rich results and machine ingestion.
  - **Sitemap** + `llms.txt` and a public read surface so crawlers and LLMs can index supply.
  - Fast, cacheable pages — consider deploying the discovery surface separately for crawl performance.
- 

## 10. Payments (deferred, seam designed now)

---

Payments enter at journey step 6 ("at some point, immediate payment"). Payments change the regulatory posture (PCI, refunds, no-show policy) and are **out of scope for v1**, but we design the seam now — a `payment_pending` Visit state and a payment provider port wired to nothing — exactly as we did with the SMS provider interface. This lets us switch it on later without a structural change.

---

## 11. Security, privacy & compliance

---

- **POPIA / consent.** Cross-practice global identity makes consent scope critical. A patient's clinical file at Practice A must be invisible to Practice B. Enforced by the identity split (Seam 1) + row-level security + the existing consent hard-block.
  - **No accounts.** Identity is the WhatsApp number; patients never log in. Access to a Visit is via `visit_id` + `access_token` (magic link), validated server-side, service-role DB access only, never direct table access from the client.
  - **Rate limiting.** Public, unauthenticated endpoints that can spend money (SMS/WhatsApp) or fill a queue must be rate-limited per-phone and per-IP.
  - **Token lifecycle.** Access tokens expire relative to the Visit date; stale tokens are rejected.
-

## 12. Delivery plan (de-risked, not big-bang)

| Phase | Deliverable                                                                                                                                                                                         | Why this order                                            |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 0     | <b>Rename</b> ClinicOS → ApIOS across package/domain/copy                                                                                                                                           | Cheap now, painful after the subtree merge                |
| 1     | <b>Seam 1 — split identity</b> (global <code>patients</code> vs. per-practice <code>practice_patients</code> )                                                                                      | Most expensive to reverse; unblocks cross-practice search |
| 2     | <b>Seam 2 — Visit state machine</b> in <code>@aplos/core</code> (adds <code>confirmed</code> , <code>arrived</code> , <code>checked_in</code> , <code>consult_ended</code> , <code>no_show</code> ) | Single source of truth for legal transitions              |
| 3     | <b>Seam 3 — event/outbox</b> + WhatsApp channel behind the provider seam                                                                                                                            | Enables WhatsApp-first steps 10–15                        |
| 4     | <b>Discovery + Booking surfaces</b> reading the same Visit the queue understands                                                                                                                    | Ships the demand-side front door                          |
| 5     | Resume <b>app-absorb</b> (subtree merge of the two apps into the monorepo)                                                                                                                          | Once the rename + seams have settled the target shape     |
| Later | Payments, calendar/PMS sync for <code>live</code> practices, realtime (websockets), multi-language                                                                                                  | After pilot pull                                          |

**Guardrail:** we are at "foundation built, app-absorb paused." Get the rename + four seams right, then add surfaces incrementally. Discovery and booking can ship reading the same Visit the queue already understands, long before the whole tree is reorganised.

## 13. Risks & mitigations

| Risk                                                                                           | Mitigation                                                                                                  |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Booking adds friction vs. the zero-friction walk-in (violates "never add friction to joining") | Keep the walk-in QR path as the zero-friction door; booking is an <i>additional</i> door, not a replacement |
| Cross-practice data leakage (POPIA)                                                            | Identity split + RLS + consent hard-block, designed in Seam 1 before any marketplace data exists            |
| WhatsApp Business API approval / template lead time                                            | Start template approval early; SMS remains the fallback via the same provider seam                          |
| Big-bang restructure destabilises current pilots                                               | Incremental sequence; rename first, seams next, surfaces last; app-absorb resumes only after                |
| SMS/WhatsApp cost as an abuse vector                                                           | Rate-limit public endpoints per-phone/per-IP; cap messages per Visit                                        |
| SEO/GEO structure churn                                                                        | Lock canonical URL + structured-data shape before inbound links exist                                       |



## 14. Out of scope (v1)

Payments (seam only), video consultations, prescriptions, full medical records beyond the existing patient file, AI triage/diagnosis, patient accounts/logins, external calendar/PMS sync (arrives with **live**-mode maturity). Each is revisited with a partner asking and a compliance review.

## 15. Open decisions

- Domain & brand finalisation: is the practice product "ApIOS", "Apollo", or "BabyBlue for Practices"?
- WhatsApp Business API provider (Meta direct vs. an aggregator such as Twilio/360dialog).
- Whether the discovery surface is a separate deployment or a route group within the patient app.
- Payment provider (when payments are switched on) — SA-local (e.g. Paystack/Yoco) vs. Stripe.
- Practitioner-level vs. practice-level queues for multi-practitioner practices.

## Appendix A — Visit state transition table

| From                             | Event                                           | To                   | Actor              |
|----------------------------------|-------------------------------------------------|----------------------|--------------------|
| —                                | book                                            | <b>booked</b>        | Patient (booking)  |
| —                                | walk_in_join                                    | <b>queued</b>        | Patient (QR)       |
| <b>booked</b>                    | confirm (live: auto; request: practice accepts) | <b>confirmed</b>     | System / Practice  |
| <b>confirmed</b>                 | window_open (T-2h)                              | <b>queued</b>        | System (promoter)  |
| <b>queued</b>                    | claim_arrival                                   | <b>arrived</b>       | Patient            |
| <b>arrived</b>                   | confirm_arrival                                 | <b>checked_in</b>    | Practice           |
| <b>checked_in</b>                | start_consult                                   | <b>in_consult</b>    | Practice           |
| <b>in_consult</b>                | end_consult                                     | <b>consult_ended</b> | Practice           |
| <b>consult_ended</b>             | confirm_complete                                | <b>completed</b>     | Patient            |
| any (pre-consult)                | cancel                                          | <b>cancelled</b>     | Patient / Practice |
| <b>queued</b> / <b>confirmed</b> | no_show                                         | <b>no_show</b>       | System / Practice  |
| <b>booked</b> / <b>confirmed</b> | expire                                          | <b>expired</b>       | System             |

## Appendix B — Glossary

- **Visit** — the central entity; one patient's single trip through a practice, from booking/walk-in to completion.
- **Front door** — a way a Visit is created: QR walk-in or online booking.

- **Promotion** — the moment a `scheduled / confirmed` booking becomes a live `queued` entry (at the 2-hour window).
- **Spine** — the shared backend + `@aplos/core` domain logic every surface depends on.
- **Surface** — a distinct client with its own rendering needs (Discovery, Booking, Presence, WhatsApp, Practice portal).
- **Seam** — an architectural boundary chosen deliberately to make future change cheap.